Lab 3: CoreMark Benchmark Report

Name: [Your Name]
Partner: [Partner Name]
Course: [Course Number]

October 31, 2025

System Information

CPU Model:	11th Gen Intel(R) Core(TM) i7-11375H @ 3.30GHz
Base Clock Frequency:	3300 MHz
Max Clock Frequency:	3302 MHz
Number of Cores:	4
Number of Logical Processors:	8
Operating System:	Windows
Compiler:	GCC version 15.2.0 (MSYS2 MinGW-w64)

Benchmark Results

Default Build Performance

Run #	Iterations/Sec
1	20655.47
2	20567.67
3	20635.58
4	20919.04
5	20664.00
Average	**20688.35**

Compiler Optimization Comparison

Optimization Flag	Run 1	Run 2	Run 3	Run 4	Run 5	Average	% Chang
-O0	5432.90	5374.51	5099.91	5235.10	5248.84	**5278.25**	**-74**
-O1	18679.95	19169.33	19353.59	19173.00	19374.84	**19150.14**	**-7**
-O2 (Default)	20655.47	20567.67	20635.58	20919.04	20664.00	**20688.35**	
-O3	21578.08	21326.51	21875.46	21811.84	20913.21	**21501.02**	**+3**

-Ofast	21423.98	21219.41	21867.48	21609.16	21941.05	**21612.22**	+4
-march=native	19692.79	20221.08	20373.51	20153.16	19646.37	**20017.38**	**-3**

Formula for % Change:

$$\text{Percent Change} = \frac{(\text{New Value} - \text{Default Value})}{\text{Default Value}} \times 100$$

Analysis of Results

Performance Observations

Impact of Optimization Levels The benchmark results demonstrate the strong influence of compiler optimization on CPU performance. The transition from `-O0` to `-Ofast` yields substantial improvements.

- **-O0 (No Optimization):** 5278 iterations/sec, a 74.49% decrease relative to `-O2`. Produces unoptimized code with inefficient memory and instruction handling.
- **-O1 (Basic Optimization):** 19150 iterations/sec, still 7.44% slower than `-O2`. Includes basic optimizations like dead code removal.
- **-O2 (Default):** 20688 iterations/sec, GCC's recommended level balancing performance and size.
- **-O3:** 3.93% higher than `-O2`, introducing loop unrolling and inlining.
- **-Ofast:** 4.46% higher than `-O2`, relaxing IEEE compliance for extra speed.

Overall, moving from `-O0` to `-O2` gives a 292% increase, while `-O2` to `-Ofast` yields diminishing returns.

Performance Variations Run-to-run variations remained under 5%, except at `-O0`, likely due to thermal and OS scheduling effects. Averaging multiple runs mitigates these variations.

Architecture-Specific Optimizations The `-march=native` flag underperformed (−3.24%), likely because CoreMark's integer-heavy workload doesn't exploit advanced vector instructions. Larger code size may also harm cache performance.

CoreMark Algorithm Explanation

1. List Processing (Linked Lists)

- **Operations:** Searching, sorting, traversal
- **Tests:** Pointer chasing, branch prediction, cache locality
- **Relevance:** Task scheduling, memory management

2. Matrix Manipulation

- **Operations:** Matrix addition, multiplication
- **Tests:** Arithmetic throughput, data locality
- **Relevance:** Control systems, DSP, graphics

3. State Machine

- **Operations:** State transitions, string parsing
- **Tests:** Branch prediction, control flow
- **Relevance:** Protocol handlers, parsers

4. CRC (Cyclic Redundancy Check)

- **Operations:** Bit-level checksum calculation
- **Tests:** Integer arithmetic, logical operations
- **Relevance:** Data integrity, communications

CoreMark Characteristics: No I/O, minimal memory use, deterministic, portable, and self-validating.

RISC-V ISA Extension Sets

RV32I - Base Integer Instruction Set

Foundation ISA with 32-bit registers, basic arithmetic, load/store, and control flow. Ideal for simple embedded systems.

RV32IM - Base + Multiply/Divide

Adds hardware multiply/divide (M extension): MUL, DIV, REM instructions. Major performance boost for arithmetic workloads.

RV32IMC - Base + Multiply/Divide + Compressed

Adds compressed (C) 16-bit instructions. Reduces code size 25–30%, improves cache utilization, no performance penalty.

RV32GC - General-Purpose Configuration

Combines IMAFD+C: integer, atomic, and floating-point support. Suitable for advanced embedded and application processors.

RV64I - 64-bit Base

Extends address space and registers to 64 bits. Adds LD/SD instructions and 32-bit word operations. Ideal for large-memory or high-performance systems.

Comparison Summary

Feature	RV32I	RV32IM	RV32IMC	RV32GC	RV64I
Multiply/Divide	Software	Hardware	Hardware	Hardware	Software (or +M)
Code Size	Baseline	Baseline	70–75%	70–75%	Larger
Floating-Point	Software	Software	Software	Hardware	Software (or +FD)
Atomics	No	No	No	Yes	Optional (+A)
Address Space	32-bit	32-bit	32-bit	32-bit	64-bit
Typical Use	Educational	Embedded	Memory-limited	General-purpose	High-performance

Conclusions

1. **Compiler Optimizations Matter:** 292% improvement from `-O0` to `-O2`. Diminishing gains beyond.
2. **Benchmarking Enables Decisions:** Data-driven CPU selection using quantitative evidence.
3. **Algorithm Awareness:** Understanding CoreMark workloads aids hardware-software co-design.
4. **RISC-V Flexibility:** Modular extensions allow tailored trade-offs between cost, power, and performance.
5. **Validate Optimizations:** `-march=native` underperformance shows testing is essential.

References

1. CoreMark Official Repository: `https://github.com/eembc/coremark`
2. EEMBC CoreMark Documentation: `https://www.eembc.org/coremark/`
3. RISC-V ISA Specifications: `https://riscv.org/technical/specifications/`
4. GCC Optimization Options: `https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html`

Appendix

Sample CoreMark Output

```
[Paste your actual CoreMark output here]
```

Build Commands Used

```
# Example:
gcc coremark.c -O2 -o coremark_O2.exe
```

CPU Information Output

```
# Paste CPU info command output here
```